UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO - UNIDAD 7

NoSQL y nuevas formas de almacenamiento

Proyecto integrador: Food Store

Modelo documental · Migración · Consultas y verificación

Materia: Base de Datos II
Carrera: Tecnicatura Universitaria en Programación (UTN)

Autores:
Lemus Nahuel
Anglat Juan
Elias Juan Cruz

Año 2026

1. Parte 1 - Modelo documental y migración

Se continúa la implementación de Food Store en PostgreSQL 18.6, base foodstore, proyecto Railway postgrefacu. Se migraron los pedidos existentes 1 a 5 con sus detalles a MongoDB 8.0.34, base foodstore_u7, colección pedidos. Es un subconjunto del mismo proyecto, no una migración completa.

Consigna 1 - Tablas, claves y correspondencia

Origen PostgreSQL	Destino MongoDB	Decisión de modelado
pedido.id (PK)	_id: Long	Conserva el identificador BIGINT y evita pedidos duplicados.
pedido.fecha, forma_pago, estado	fecha, fecha_origen_iso, forma_pago, estado	Fecha BSON en UTC y texto ISO para conservar microsegundos.
pedido.cliente_id → cliente.id	cliente: {id, alias}	Conserva la FK de origen como referencia; alias de laboratorio sin datos personales.
detalle_pedido.pedido_id → pedido.id	lineas: [...]	Se embeben las líneas, que se consultan junto con el pedido.
detalle_pedido.producto_id → producto.id	lineas.producto_id, nombre	ID del producto y nombre del catálogo; no se altera su identidad.
detalle_pedido.id, cantidad, precio_unitario	detalle_id, cantidad, precio_unitario	IDs y precio histórico conservados; subtotal y total se calculan.

Consigna 2 - Adaptación del estado y procedencia de los datos

La versión real de pedido no tenía estado. Con autorización se agregó estado text nullable en la misma base foodstore, con CHECK de valores permitidos. Solo los cinco pedidos seleccionados recibieron estados de prueba; los otros 19.995 quedaron en NULL. Estos estados no son históricos recuperados. Se conservaron fechas, importes, cantidades e identificadores ya existentes.

Pedido	Estado de prueba	Cliente (ID)	Producto (ID)	Cantidad	Precio histórico	Total
1	PENDIENTE	2	2	2	101,00	202,00
2	CONFIRMADO	3	3	3	102,00	306,00
3	ENTREGADO	4	4	4	103,00	412,00
4	CANCELADO	5	5	5	104,00	520,00
5	CONFIRMADO	6	6	1	105,00	105,00

Cada pedido de esta muestra tiene una línea. Se utilizan clientes seudonimizados (ID y alias), sin nombres, correos ni teléfonos. La cancelación de prueba no borra el pedido ni cambia sus líneas; la consulta por total incluye todos los estados.

Consigna 3 - Integridad y elección de documentos embebidos

Regla 1: cantidades enteras mayores que cero y precios históricos no negativos. Se comprueban al exportar y mediante el validador de la colección. Regla 2: conservar IDs, cantidades y precios del origen; cada subtotal es cantidad × precio_unitario y el total es la suma de las líneas. Se usan NUMERIC y Decimal128, sin convertir los importes a flotantes en la migración.

Se embeben cliente mínimo y líneas porque el acceso frecuente es al pedido completo. El precio histórico pertenece a la venta; no se sustituye por producto.precio. El catálogo sigue en PostgreSQL. La referencia cliente.id conserva trazabilidad, pero no crea una FK automática en MongoDB: la comprobación corresponde al proceso de migración.

2. Implementación y evidencia de la colección

Respaldo y preservación del origen

Antes del cambio se generó y validó el respaldo custom foodstore_antes_u7_20261027T164051Z.dump (328069 bytes). La validación utilizó pg_restore --list. Dentro de la transacción se compararon hashes SHA-256 de las columnas previas de las cinco tablas de negocio: sin diferencias. Esta comparación excluye expresamente el nuevo campo estado. No se realizó un ensayo adicional de restauración para U7.

Tabla de origen	Filas verificadas	Contenido previo al cambio
pedido	20000	SHA-256 antes = después
detalle_pedido	20000	SHA-256 antes = después
cliente	5000	SHA-256 antes = después
producto	1000	SHA-256 antes = después
categoria	20	SHA-256 antes = después

Migración ejecutada y controles

laboratorio_u7.py leyó PostgreSQL con una transacción REPEATABLE READ de solo lectura y escribió los cinco documentos mediante PyMongo insert_many(). Las credenciales se obtuvieron de Railway en memoria. MongoDB se conectó por un túnel SSH; los documentos exportados no contienen datos personales reales.

La colección usa validación estricta: campos esenciales requeridos, _id Long, fecha Date, estado limitado a cuatro valores, al menos una línea, cantidad entera positiva y precio decimal no negativo. Hay índices sobre estado, lineas.producto_id y total. El total derivado se comprueba en el programa; el validador no impone por sí mismo su igualdad con la suma.

Documento 1 - representación legible de la salida BSON

```
{
  _id: Long("1"),
  fecha: ISODate("2026-09-21T15:53:34.697Z"),
  fecha_origen_iso: "2026-09-21T15:53:34.697461+00:00",
  forma_pago: "TARJETA", estado: "PENDIENTE",
  cliente: { id: Long("2"), alias: "Cliente laboratorio 2" },
  lineas: [{ detalle_id: Long("1"), producto_id: Long("2"),
    nombre: "Producto 2", cantidad: 2,
    precio_unitario: Decimal128("101.00"),
    subtotal: Decimal128("202.00") }],
  total: Decimal128("202.00"),
  origen: { base: "foodstore", esquema: "public",
    estado_sintetico_u7: true, cliente_seudonimizado: true }
}
```

Salida real del orquestador - 27/10/2026, 13:44:24 (UTC-03:00)

```
documentos: 5
base_destino: foodstore_u7 | coleccion: pedidos
documentos_identicos_al_origen_transformado: true
coinciden: true
indices: _id_, idx_estado_u7, idx_producto_u7, idx_total_u7
```

La comprobación compara los documentos leídos de MongoDB con la exportación de PostgreSQL normalizada a BSON. BSON Date tiene precisión de milisegundos; fecha_origen_iso conserva los microsegundos originales. Evidencia completa: pedidos_migrados.ejson, preparacion_postgresql.json y verificacion.json, adjuntas al paquete.

3. Parte 2 - Consultas y verificación entre motores

Consigna 4 - Cuatro consultas MongoDB

El orquestador ejecutó los siguientes filtros con PyMongo y ordenó por _id. Las instrucciones equivalentes para mongosh se incluyen en consultas_mongodb.js. En todas las consultas se utiliza la misma muestra, pedidos 1 a 5; no se compara una colección de cinco documentos con los 20.000 pedidos completos del origen.

```
const c = db.getSiblingDB("foodstore_u7").pedidos;
// a) Mostrar todos los pedidos, ID y estado.
c.find({}, {_id:1, estado:1}).sort({_id:1});
// b) Estado existente en la muestra de prueba.
c.find({estado:"CONFIRMADO"}, {_id:1}).sort({_id:1});
// c) Producto dentro del arreglo de lineas.
c.find({"lineas.producto_id":Long("2")}, {_id:1}).sort({_id:1});
// d) Total superior al umbral seleccionado.
c.find({total:{$gt:Decimal128("300.00")}},
       {_id:1, total:1}).sort({_id:1});
```

Consulta	Salida de la ejecución	Cantidad
a) Todos (ID, estado)	1 PENDIENTE; 2 CONFIRMADO; 3 ENTREGADO; 4 CANCELADO; 5 CONFIRMADO	5
b) Estado CONFIRMADO	IDs: 2, 5	2
c) Producto 2	ID: 1	1
d) Total > 300,00	2: 306,00; 3: 412,00; 4: 520,00	3

Umbral: 300,00 separa los importes 105,00 y 202,00 de 306,00, 412,00 y 520,00. Permite comprobar inclusión y exclusión; no es una regla comercial de Food Store. Producto: el ID 2 proviene de la primera línea del pedido 1.

Consigna 5 - Consultas equivalentes en PostgreSQL

```
-- b) Estado; mismo subconjunto que MongoDB.
SELECT id FROM public.pedido
WHERE id IN (1,2,3,4,5) AND estado='CONFIRMADO' ORDER BY id;

-- c) Producto; DISTINCT evita duplicar pedidos al hacer JOIN.
SELECT DISTINCT p.id
FROM public.pedido p
JOIN public.detalle_pedido d ON d.pedido_id=p.id
WHERE p.id IN (1,2,3,4,5) AND d.producto_id=2 ORDER BY p.id;
```

Consulta	Cant. PostgreSQL	Cant. MongoDB	IDs PG / MongoDB	Coinciden
Estado CONFIRMADO	2	2	2, 5 / 2, 5	Sí
Producto 2	1	1	1 / 1	Sí
Todos (control adicional)	5	5	1, 2, 3, 4, 5 / iguales	Sí
Total > 300 (control adicional)	3	3	2, 3, 4 / 2, 3, 4	Sí

Resultado: coinciden cantidades e identificadores; no se detectaron diferencias en las cuatro consultas. La adaptación de nombres responde al esquema real: detalle_pedido, pedido_id, producto_id y precio_unitario. El precio proviene del detalle histórico, no del catálogo actual.

La comparación acredita equivalencia funcional en esta muestra; no mide rendimiento ni demuestra escalabilidad. No se atribuyen ventajas de tiempo a un motor sin un benchmark representativo.

4. Parte 3 - Evaluación de tecnología e IA

Consigna 6 - Consulta y respuesta de asistencia

Herramienta: Codex, utilizada durante la preparación de esta entrega. La siguiente consulta y síntesis documentan la asistencia efectiva; no se atribuye esta intervención a otras herramientas ni se presenta como revisión humana del equipo.

Consulta: Food Store usa PostgreSQL con pedido(id, fecha, forma_pago, cliente_id, estado), detalle_pedido(pedido_id, producto_id, cantidad, precio_unitario), cliente y producto. Consultamos frecuentemente pedidos completos y debemos conservar la coherencia de cantidades y precios históricos. Compará PostgreSQL y MongoDB para almacenar y consultar estos pedidos, explicá el papel de documentos embebidos y reglas de integridad, y recomendá una alternativa sin suponer mejoras de rendimiento no medidas.

Respuesta sintetizada: MongoDB permite embeber las líneas y los datos mínimos del cliente para recuperar un pedido en una sola lectura documental, evitando reconstruirlo con JOIN. Su esquema flexible no impide validar tipos y rangos, pero esas reglas deben configurarse y no reemplazan por sí solas las relaciones del modelo SQL. PostgreSQL conserva ventajas para la coherencia entre pedidos, productos y stock mediante claves, restricciones y transacciones. Conviene mantenerlo como fuente de verdad y evaluar MongoDB como proyección de lectura solo si el acceso a pedidos completos justifica la sincronización adicional. Cinco pedidos permiten verificar equivalencia, pero no probar una mejora de velocidad.

Consigna 7 - Contraste de dos afirmaciones concretas

Afirmación de la respuesta	Contraste y evidencia	Evaluación
Embeber las líneas permite recuperar un pedido en una sola lectura documental.	MongoDB documenta la recuperación de datos relacionados en una operación [1]. El documento 1 contiene cliente y lineas, sin JOIN. La prueba verifica estructura, no una aceleración.	Respaldada para este acceso. El documento tiene que respetar límites de tamaño.
El esquema flexible permite configurar validaciones de tipos y rangos.	La documentación explica schema validation y rechazo de documentos inválidos [2]. La colección se creó con validación strict/error, cantidades positivas y precios no negativos.	Respaldada. La suma de las líneas se valida en el programa, no en el JSON Schema.

Conclusión - menos de 200 palabras

Conviene mantener PostgreSQL como fuente de verdad de Food Store. El modelo actual ya relaciona clientes, productos, pedidos y detalles mediante claves y restricciones; conservar esa coherencia es importante al registrar ventas y actualizar existencias. Además, NUMERIC permite operar importes exactos sin reemplazar precios históricos por precios actuales del catálogo. MongoDB resulta adecuado para representar un pedido completo con sus líneas embebidas y simplificar su lectura. La migración de cinco pedidos y la igualdad de los resultados muestran que ese subconjunto puede expresarse correctamente en ambos motores. Sin embargo, no se midieron tiempos ni cargas representativas, por lo que no se justifica reemplazar el motor relacional por una supuesta mayor velocidad. Una combinación sería razonable si un caso real de consulta frecuente lo requiere, siempre definiendo cómo sincronizar la proyección documental y evitar inconsistencias. Para el alcance actual, MongoDB queda como laboratorio verificado; PostgreSQL conserva la operación principal.

Referencias técnicas y archivos adjuntos

[1] MongoDB - Embedded Data: https://www.mongodb.com/docs/manual/data-modeling/embedding/
[2] MongoDB - Schema Validation: https://www.mongodb.com/docs/manual/core/schema-validation/
[3] PostgreSQL 18 - Numeric Types: https://www.postgresql.org/docs/18/datatype-numeric.html

Repositorio: github.com/hero21lb/foodstore-gestor. Carpeta: food-store-tp-u7. Adjuntos: consultas_mongodb.js, consultas_postgresql.sql, laboratorio_u7.py, requisitos y evidencias seudonimizadas. Consulta documental: 27/10/2026. No se adjuntan dumps ni credenciales.